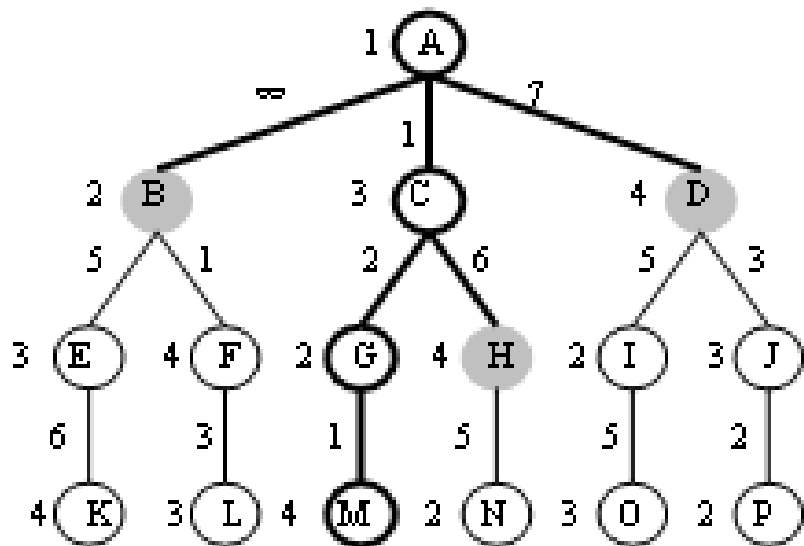


- 有4个城市的旅行商问题，其费用矩阵如图所示，矩阵的每个元素 a_{ij} 分别为相应 i 到 j 城市的旅费代价， ∞ 表示无意义或无通路。求从第一个城市出发，最后回到城市1的最短路线。请画出解空间树，给出最优解。

∞	∞	1	7
8	∞	5	1
7	2	∞	6
2	5	3	∞

- 有4个城市的旅行商问题，其费用矩阵如图所示，矩阵的每个元素 a_{ij} 分别为相应 i 到 j 城市的旅费代价， ∞ 表示无意义或无通路。求从第一个城市出发，最后回到城市1的最短路线。请画出解空间树，给出最优解。

∞	∞	1	7
8	∞	5	1
7	2	∞	6
2	5	3	∞



通过键盘输入一个高精度的正整数 n (n 的有效位数 ≤ 240)，去掉其中任意 s 个数字后，剩下的数字按原左右次序将组成一个新的正整数。编程对给定的 n 和 s ，寻找一种方案，使得剩下的数字组成的新数最小，并分析算法的最优性。例：输入：178543， $S=4$ ；输出：13。

解：为了尽可能地逼近目标，选取的贪心策略为：每一步总是选择一个使剩下的数最小的数字删去，即按高位到低位的顺序搜索，若各位数字递增，则删除最后一个数字，否则删除第一个递减区间的首字符。然后回到串首，按上述规则再删除下一个数字。重复以上过程 s 次，剩下的数字串便是问题的解了。贪心选择性质，可以由替换法证明。最优子结构由反证法证明。

具体算法如下：

输入 s, n ;

while ($s > 0$)

{ $i=1$; //从串首开始找

while ($i < \text{length}(n)$) && ($n[i] < n[i+1]$)

{ $i++$;}

delete($n, i, 1$); //删除字符串 n 的第 i 个字符

$s--$; }

while ($\text{length}(n) > 1$) && ($n[1] = '0'$)

delete($n, 1, 1$); //删去串首可能产生的无用零

输出 n ;

2. 考虑求下列数列的通项 a_n : $a_0=0$, $a_1=1$, $a_n=5a_{n-1}-6a_{n-2}$ ($n>1$)。下面是一个递归算法。请重新设计算法使得用 $O(n)$ 时间求出 $A(n)$ 。(6分)

```
int A(int n)
{
    if (n==0) return 0;
    if (n==1) return 1;
    else return 5*A(n-1)-6*A(n-2);
}
```

解:该算法的特点:子问题的求解有大量的重复。可以改进如下:根据数列的通项公式,采用自下而上来计算: $f(2)=5f(1)-6f(0)$, $f(3)=5f(2)-6f(1)$, ..., $f(n)=5f(n-1)-6f(n-2)$ 。此时算法用 $O(n)$ 时间即可求出 $A(n)$ 。

```
int A(n);
A(0)=0;
A(1)=1;
for i=2:n
    A(i)=A(i-1)*5-A(i-2)*6
end
```

给定 n 个整数的数组 $A[1,...,n]$ ，以及一个整数 x ，设计一个分治算法，求出 x 在数组 A 中的出现的次数，并分析所设计算法的时间复杂度。

解：a、先将问题划分为大小近似相等的两个子问题

b、对子问题递归调用该算法进行处理，递归出口为子问题只含一个元素，若该元素等

于 x ，则返回 x 的出现次数 $\square$ ，若该元素不等于 x ，则返回 $-$

c、原问题结果为这两个子问题所得结果之和

Countx(A, p, r, x)

1 **if** $p=r$ **then**

2 **if** $A[p]=x$ **then**

3 **return** 1

4 **else**

5 **return** 0

6 **else**

7 $q \leftarrow (p+r)/2$

8 **return** (Countx(A, p, q, x)+Countx($A, q+1, r, x$))

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1 \\ 2T(\lfloor n/2 \rfloor) + \Theta(1) & \text{if } n \geq 2. \end{cases}$$

有 N 件物品和一个背包。背包容量为 c ，容积为 d 。第 i 件物品的重量是 $c[i]$ ，价值是 $w[i]$ ，体积是 $d[i]$ 。将哪些物品装入背包可使这些物品的重量总和不超过背包容量，体积不超过背包体积，且价值总和最大？请证明使用动态规划算法求解这个问题能获得最优解，写出递归公式。分析算法复杂度。

该问题是二维 0-1 背包问题。问题的形式化描述是：给定 $c > 0, d > 0, w_i > 0, b_i > 0, v_i > 0, 1 \leq i \leq n$ ，要求找出 n 元 0-1 向量 $(x_1, x_2, \dots, x_n), x_i \in \{0, 1\}, 1 \leq i \leq n$ ，使得 $\sum_{i=1}^n w_i x_i \leq c$ ， $\sum_{i=1}^n b_i x_i \leq d$ 而且 $\sum_{i=1}^n v_i x_i$ 达到最大。

因此，二维 0-1 背包问题也是一个特殊的整数规划问题。

$$\begin{aligned} & \max \sum_{i=1}^n v_i x_i \\ & \begin{cases} \sum_{i=1}^n w_i x_i \leq c \\ \sum_{i=1}^n b_i x_i \leq d \\ x_i \in \{0, 1\}, 1 \leq i \leq n \end{cases} \end{aligned}$$

容易证明该问题具有最优子结构性质。

设所给二维 0-1 背包问题的子问题

$$\begin{aligned} & \max \sum_{t=i}^n v_t x_t \\ & \begin{cases} \sum_{t=i}^n w_t x_t \leq j \\ \sum_{t=i}^n b_t x_t \leq k \\ x_t \in \{0, 1\}, i \leq t \leq n \end{cases} \end{aligned}$$

的最优值为 $m(i, j, k)$ ，即 $m(i, j, k)$ 是背包容量为 j ，容积为 k ，可选择物品为 $i, i+1, \dots, n$ 时二维 0-1 背包问题的最优值。由二维 0-1 背包问题的最优子结构性质，可以建立计算 $m(i, j, k)$ 的递归式如下：

$$m(i, j, k) = \begin{cases} \max\{m(i+1, j), m(i+1, j-w_i, k-b_i) + v_i\} & j \geq w_i \text{ and } k \geq b_i \\ m(i+1, j) & 0 \leq j < w_i \text{ or } 0 \leq k < b_i \end{cases}$$

$$m(n, j, k) = \begin{cases} v_n & j \geq w_n \text{ and } k \geq b_n \\ 0 & 0 \leq j < w_n \text{ or } 0 \leq k < b_n \end{cases}$$

按此递归式计算出的 $m(n, c, d)$ 为最优值。算法所需的计算时间为 $O(ncd)$ 。

定义0/1/2背包问题为： $\max \sum_{i=1}^n v_i x_i$ ，使得 $\sum_{i=1}^n w_i x_i \leq W$ ，其中 $x_i \in \{0,1,2\}$ ，设其子问

题的定义类似0/1背包问题中的定义，完成下列任务。

- 1) 从0/1/2背包问题中推出类似于0/1背包问题的递归方程。
- 2) 假设 w_i, v_i, W 为整数，设计一个类似于0/1背包的动态规划求解算法，并构造出最优解。
- 3) 分析上述算法的时间复杂度。

$p[i, w]$: 当背包容量为 w 时，可以从物品1到 i 获得的最大价值

(1)不装入物品 i $p[i, w]=p[i-1, w]$

(2)装入1个物品 i $p[i, w]=p[i-1, w-w_i]+v_i$

(3)装入2个物品 i $p[i, w]=p[i-1, w-2w_i]+2v_i$

递归方程为：

$p[i, w]=\max \{p[i-1, w], p[i-1, w-w_i]+v_i, p[i-1, w-2w_i]+2v_i\}$

012knapsack(W, v, w)

1 **for** $i \leftarrow 2$ to n **do**

2 **for** $w \leftarrow 0$ to W **do**

3 **if** $2w_i \leq w$ **then**

4 $p[i, w] \leftarrow \max \{p[i-1, w], p[i-1, w-w_i]+v_i, p[i-1, w-2w_i]+2v_i\}$

5 **else if** $w_i \leq w$ **then**

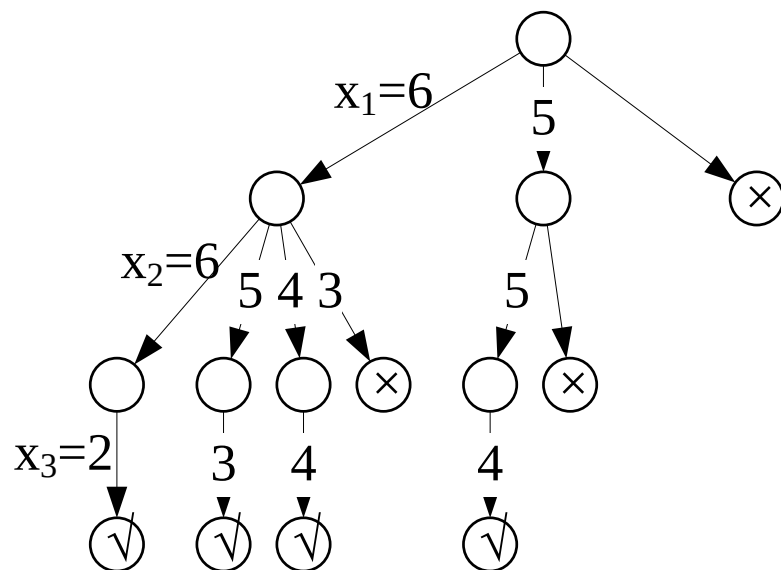
6 $p[i, w] \leftarrow \max \{p[i-1, w], p[i-1, w-w_i]+v_i\}$

7 **else**

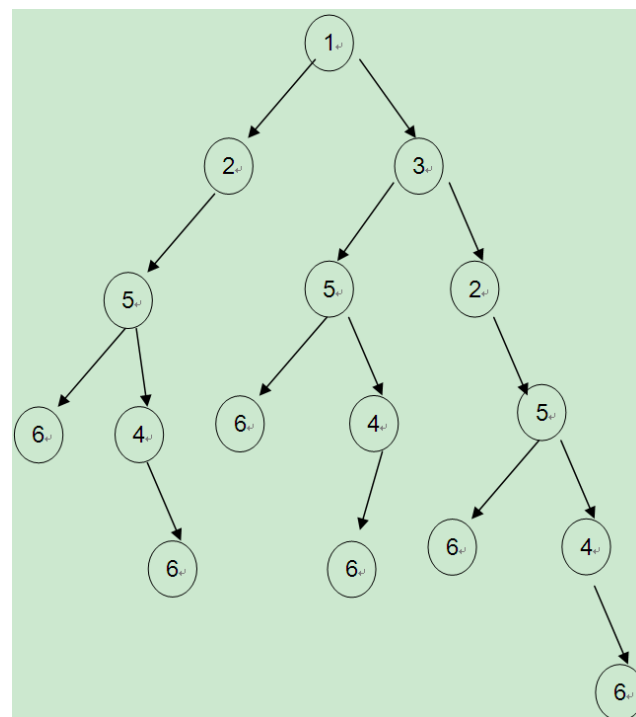
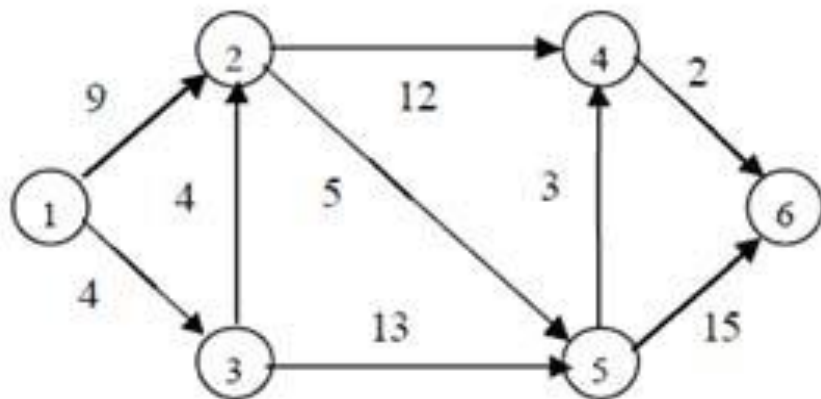
8 $p[i, w] \leftarrow p[i-1, w]$

设 x_1 、 x_2 、 x_3 是一个三角形的三条边，而且 $x_1+x_2+x_3=14$ 。请问有多少种不同的三角形？给出解答过程。要求每条边的边长均为整数。

解：由于 x_1 、 x_2 、 x_3 是三角形的三条边，从而 $x_i+x_j>x_k$ ， $|x_i-x_j|<x_k$ ，（ $i,j,k=1,2,3$ ），根据 $x_1+x_2+x_3=14$ 可知 $1<x_i<7$ （ $i=1,2,3$ ）。利用回溯法求解得到：



请设计**两种算法**找到下图中的最短路径，用**图表的方式**给出解决问题的过程。如果是用搜索算法，请画出搜索树。说明相关算法的类型。



Dijkstra算法:

迭代次数	集合 X 中的元素	$\lambda[1]$	$\lambda[2]$	$\lambda[3]$	$\lambda[4]$	$\lambda[5]$	$\lambda[6]$	最短路径
初始化	1	0	9	<u>4</u>	∞	∞	∞	
1	1, 3	0	<u>8</u>	4	∞	17	∞	1→3
2	1, 3, 2	0	8	4	20	<u>13</u>	∞	1→3→2
3	1, 3, 2, 5	0	8	4	<u>16</u>	13	28	1→3→2→5
4	1, 3, 2, 5, 4	0	8	4	16	13	<u>18</u>	1→3→2→5→4
5	1, 3, 2, 5, 4, 6	0	8	4	16	13	18	1→3→2→5→4→6

给出一个分治算法来找出 n 个元素的序列中的第2大元素，并分析算法的时间复杂度。

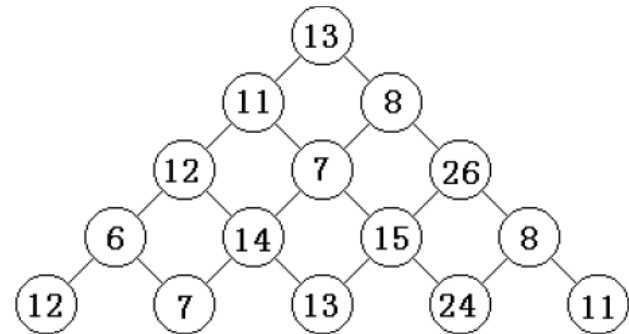
解：当序列 $A[1..n]$ 中元素的个数 $n=2$ 时，通过直接比较即可找出序列的第2 大元素。当 $n>2$ 时，先求出序列 $A[1..n-1]$ 中的第1 大元素 x_1 和第2 大元素 x_2 ；然后，通过2次比较即可在三个元素 x_1 ， x_2 和 $A[n]$ 中找出第2 大元素，该元素即为 $A[1..n]$ 中的第2 大元素。

```
SecondElement(A[low..high],max1,max2)
{ //假设主程序中调用该过程条件为high-low>=2
  if(hight-low==2)
    { if(A[low]<A[hight]){max2= A[low];max1=A[hight];}
      else {max2= A[hight];max1=A[low];}
    }
  else{
    SecondElement(A[low..high],x1,x2);
    if(x1<=A[n]) { max2=max1; max1=A[n];}
    else if(x2>=A[n]) { max2=x2; max1=x1; }
    else { max2=A[n]; max1=x1; }
  }
}
```

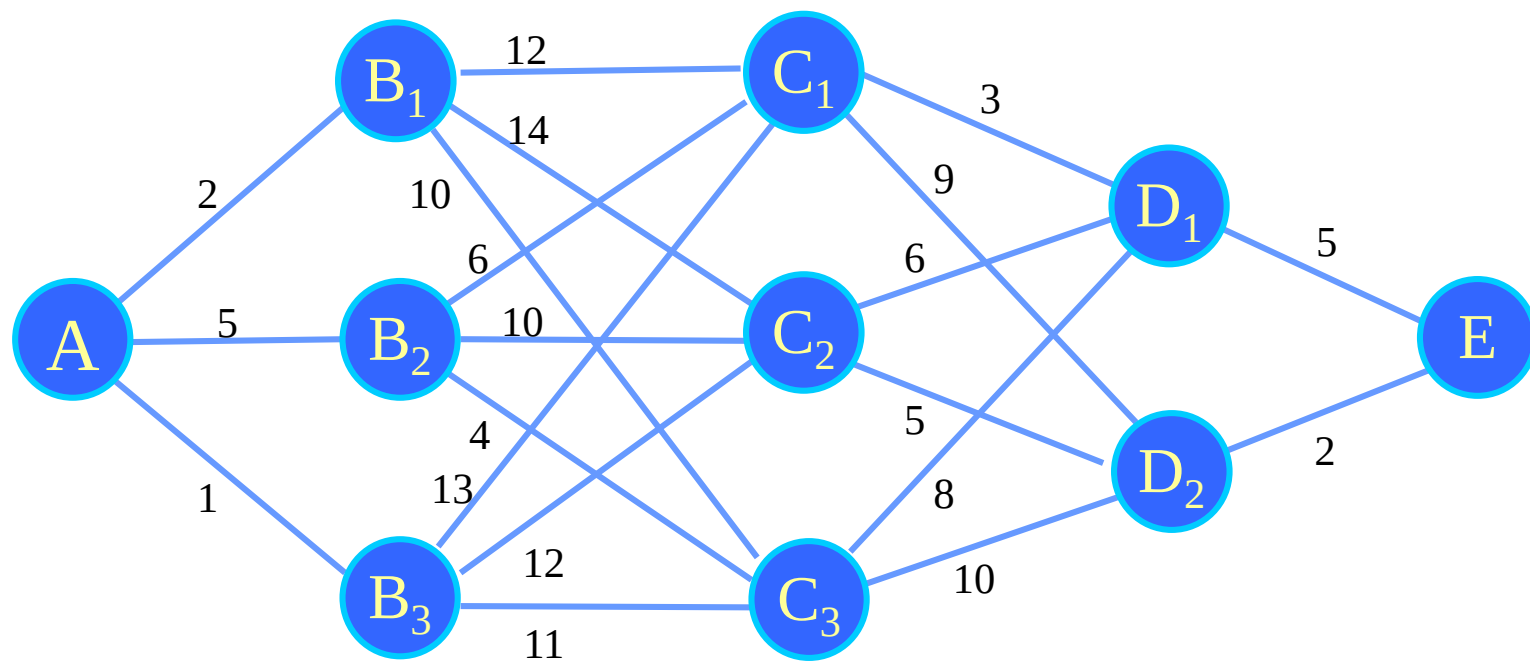
$T(n)=T(n-1)+2$; $T(2)=1$ 。解得 $T(n)=2n-3$ 。

数塔问题。有形如下图所示的数塔，从顶部出发，在每一结点可以选择向左走或是向右走，一起走到底层，要求找出一条路径，使路径上的值最大。

```
for(r=n-2;r>=0;r--) //自底向上递归计算
for(c=0;__ 1 __;c++)
    if( t[r+1][c]>t[r+1][c+1]) __ 2 __;
    else __ 3 __ ;
```



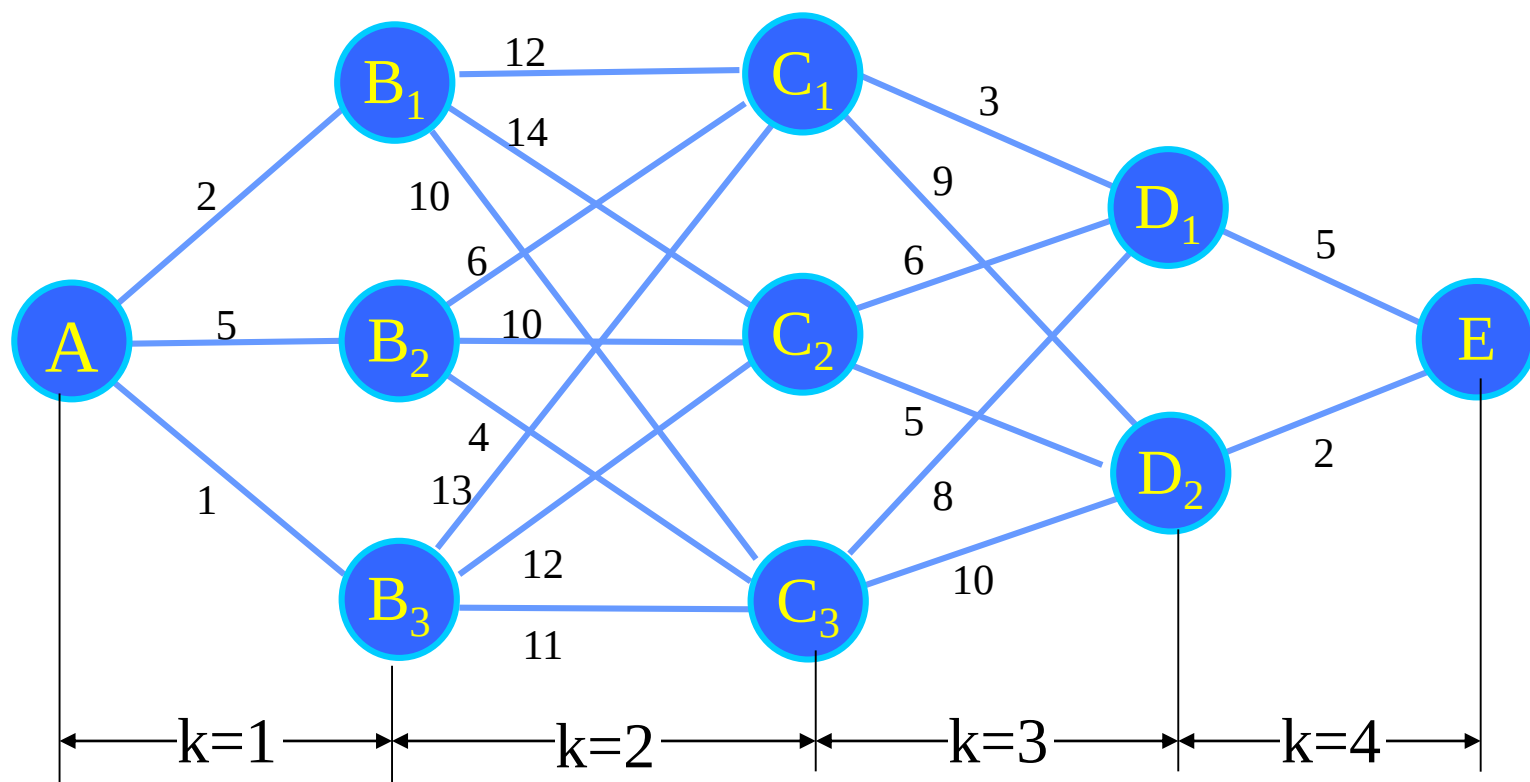
```
c<=r;
t[r][c]+=t[r+1][c];
t[r][c]+=t[r+1][c+1]
```



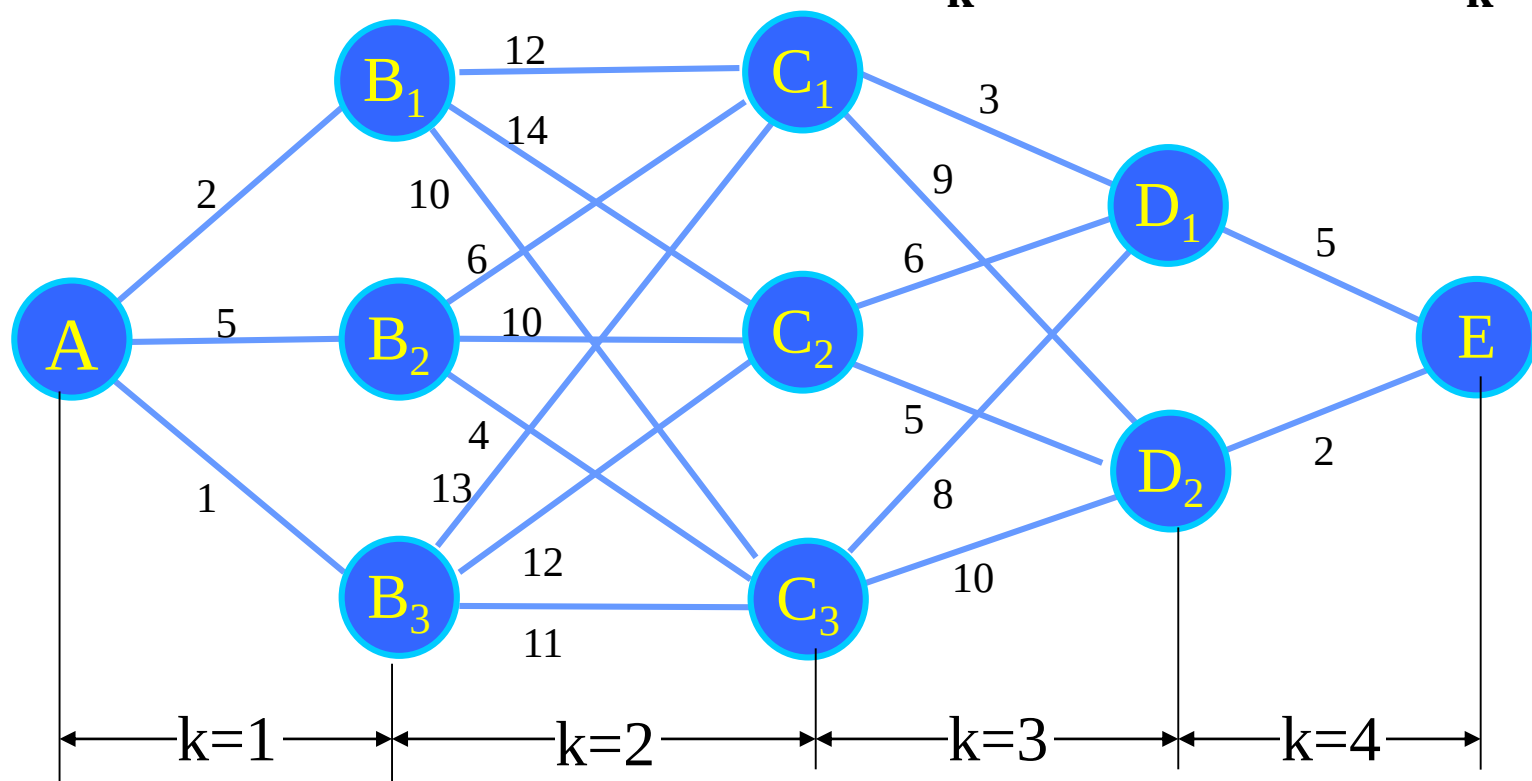
求从A到E的最短路径和最短距离

解（一）建立动态规划逆序递推基本方程

1、划分阶段：将原问题按照空间划分为四个阶段，即 $n=4$ ，阶段变量 $k=1,2,3,4$



2、定义状态变量 s_k ： 令 s_k 表示各阶段的起始位置，则各阶段的状态变量 s_k 及其状态集合 S_k 为：



当 $k=1$ 时, $s_1=A$

$S_1=\{A\}$

当 $k=2$ 时, $s_2= B_1$ 或 B_2 或 B_3

$S_2=\{B_1, B_2, B_3\}$

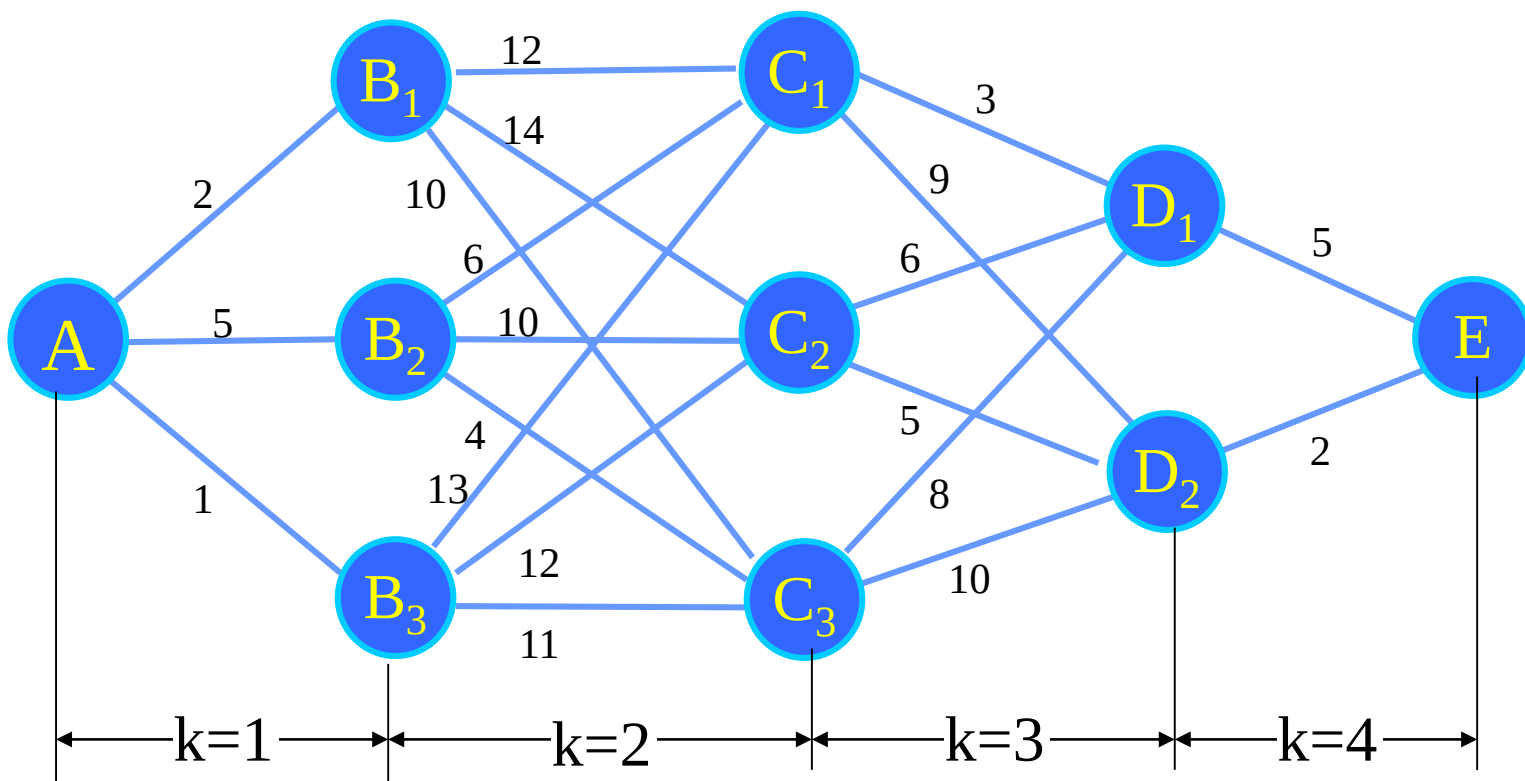
当 $k=3$ 时, $s_3= C_1$ 或 C_2 或 C_3

$S_3=\{C_1, C_2, C_3\}$

当 $k=4$ 时, $s_4= D_1$ 或 D_2

$S_4=\{D_1, D_2\}$

3、定义决策变量 $x_k(s_k)$ ： 令 $x_k(s_k)$ 表示从阶段 k 的 s_k 出发所做的选择。



各阶段的决策变量 $x_k(s_k)$ 及其允许决策集合 $D_k(s_k)$ 为:

$$x_1(s_1) : \quad x_1(A) = B_1 \text{ 或 } B_2 \text{ 或 } B_3 \qquad D_1(A) = \{B_1, B_2, B_3\}$$

$$x_2(s_2) : \quad x_2(B_1) = C_1 \text{ 或 } C_2 \text{ 或 } C_3 \qquad D_2(B_1) = \{C_1, C_2, C_3\}$$

$$x_2(B_2) = C_1 \text{ 或 } C_2 \text{ 或 } C_3 \qquad D_2(B_2) = \{C_1, C_2, C_3\}$$

$$x_2(B_3) = C_1 \text{ 或 } C_2 \text{ 或 } C_3 \qquad D_2(B_3) = \{C_1, C_2, C_3\}$$

$$x_3(s_3) : \quad x_3(C_1) = D_1 \text{ 或 } D_2 \qquad D_3(C_1) = \{D_1, D_2\}$$

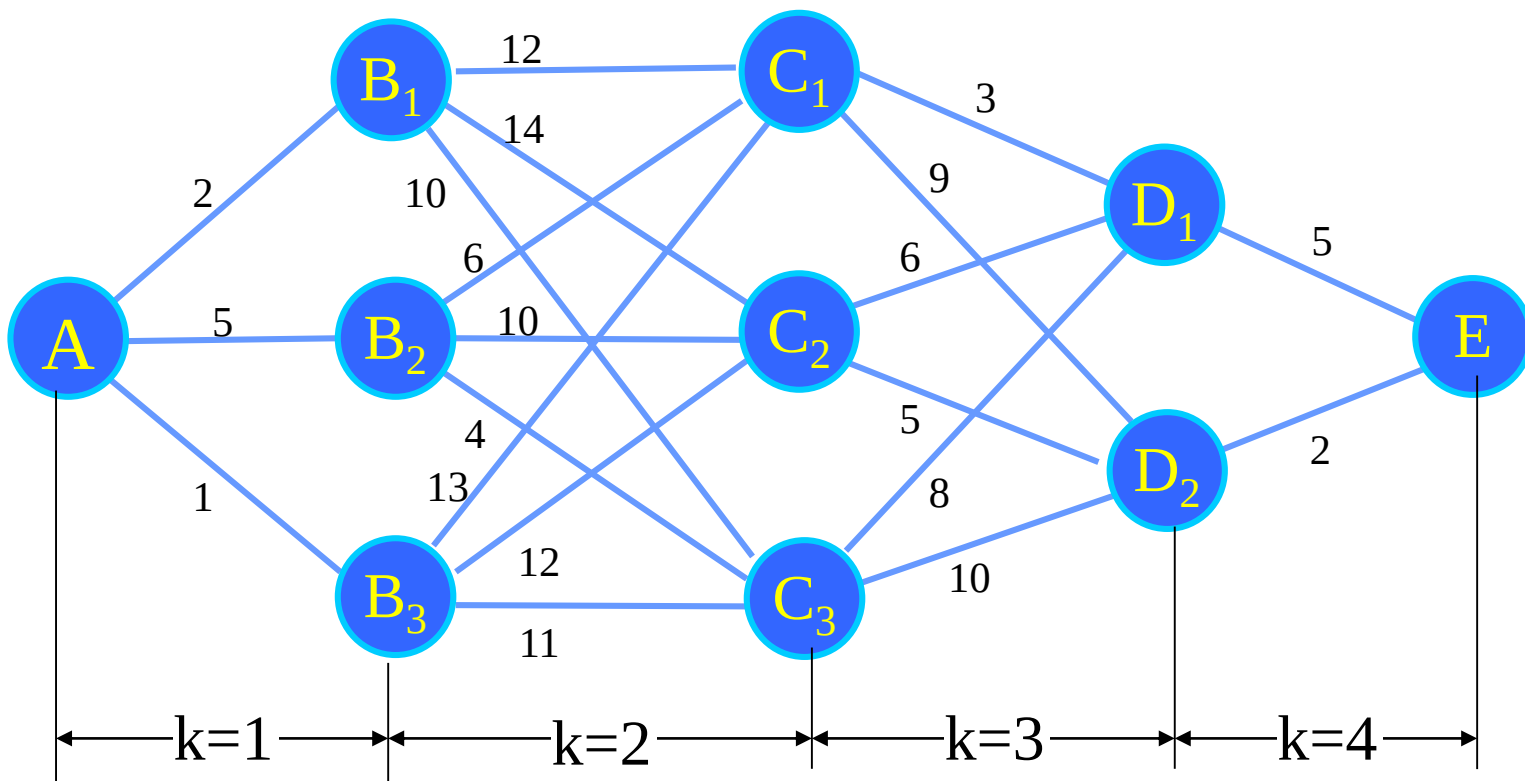
$$x_3(C_2) = D_1 \text{ 或 } D_2 \qquad D_3(C_2) = \{D_1, D_2\}$$

$$x_3(C_3) = D_1 \text{ 或 } D_2 \qquad D_3(C_3) = \{D_1, D_2\}$$

$$x_4(s_4) : \quad x_4(D_1) = E \qquad D_4(D_1) = \{E\}$$

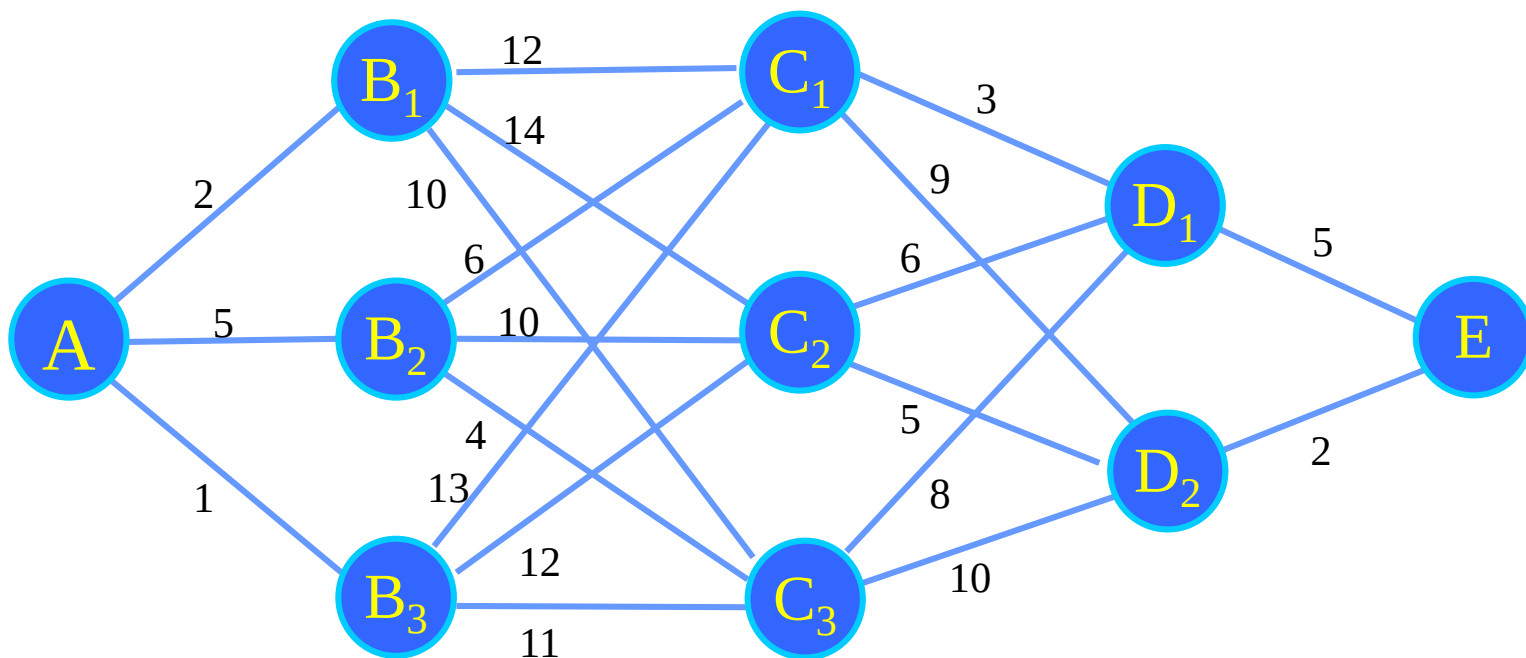
$$x_4(D_2) = E \qquad D_4(D_2) = \{E\}$$

4、确定状态转移方程：



状态转移方程： $s_{k+1} = x_k$

5、定义指标函数：

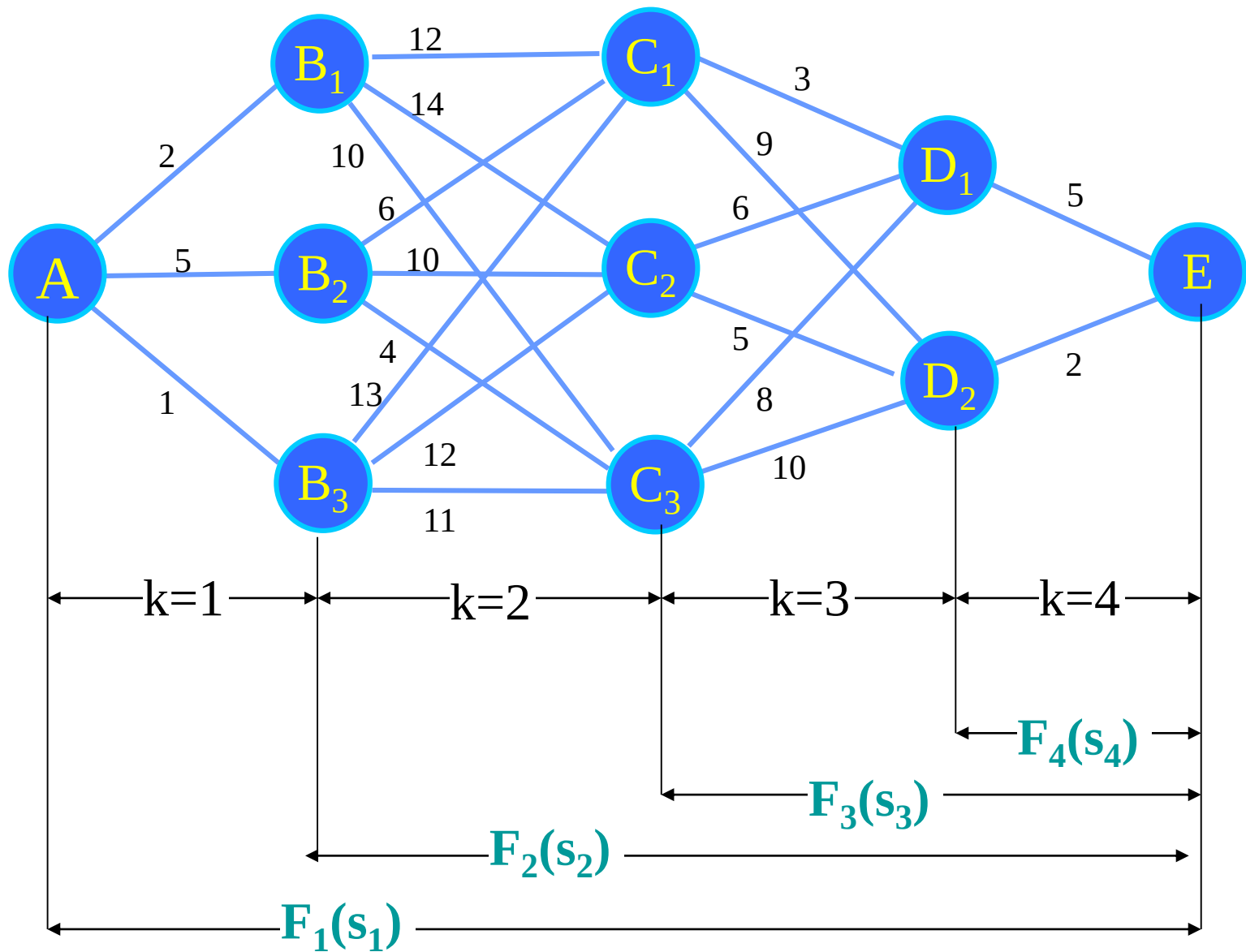


阶段指标：令 $d_k(s_k, x_k)$ ---表示相邻两个城市之间的距离

最优指标函数：

$f_k(s_k)$ ---表示从第k阶段的 s_k 到终点E的最短距离；

$f_1(s_1)$ 即 $f_1(A)$ ---表示从A到终点E的最短距离；



6、建立逆序递推基本方程：

动态规划的逆序递推算法的基本方程为：

$$\begin{cases} f_k(s_k) = \min_{x_k \in D_k(s_k)} \{d_k(s_k, x_k) + f_{k+1}(s_{k+1})\} & k=4,3,2,1 \\ f_5(s_5)=0 \end{cases}$$

从边界条件出发逐段求得原问题的最短距离